



Marwadi
University
Marwadi Chandarana Group



Experiment – 2 Aim :

Implementation and Time analysis of linear and binary search algorithm.

Hardware Required:

Mouse, keyboard **Software Required:**

code Blocks

a) Linear search Algorithm:



step 1+ start
step 2+ create an array arr[100] and variables size and key
step 3+ enter size of the array
step 4+ enter array elements & print
step 5+ sort the array using Bubble sort & print sorted array.
step 6+ enter the element to be searched & store in key.
step 7+ set low=0
 high = size-1
 And calculate mid = (low+high)/2
step 8+ compare key with arr[mid]
 → if key == arr[mid] → element found
 → if key < arr[mid] → set high = mid-1
 → if key > arr[mid] → set low = mid+1
 → Repeat until low <= high
step 9+ if low > high, print element not found

Program:

```
#include <iostream>
using namespace std;
```

```
int main() {
```

Name: D.himabindu
Enrollment number :
92400118934



```
int n, target;

cout << "Enter the number of elements: ";
cin >> n;

int arr[n];

cout << "Enter " << n << " elements: ";
for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

cout << "Enter the element to search: ";
cin >> target;

// Linear Search
for (int i = 0; i < n; i++) {
    if (arr[i] == target) {
        cout << "Element found at index " << i << endl;
        return 0;
    }
}

cout << "Element not found" << endl;

return 0;
}
```

Output:

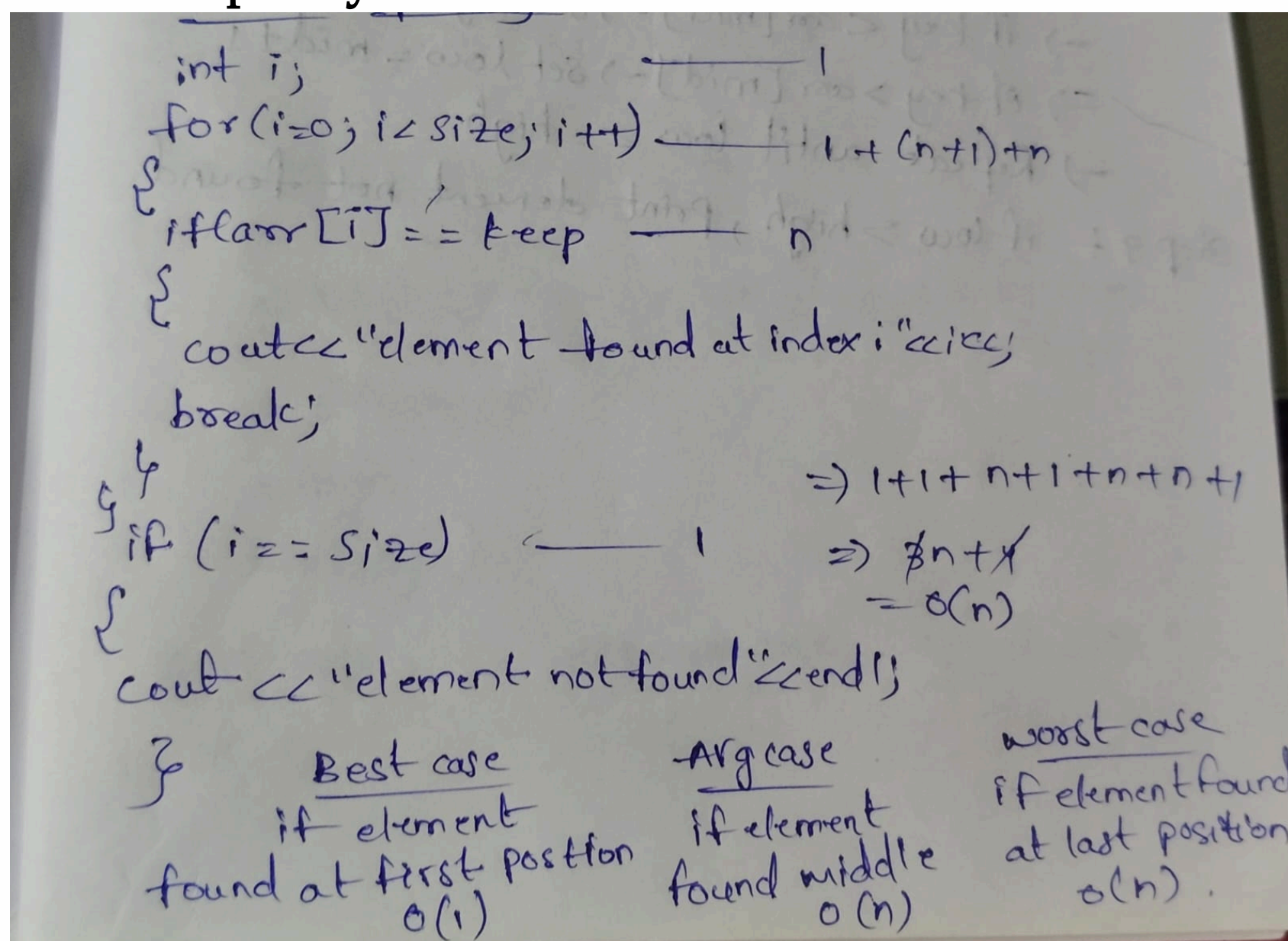
Output

```

Enter the number of elements: 4
Enter 4 elements: 10 20 43 21
Enter the element to search: 20
Element found at index 1
  
```

=== Code Execution Successful ===

Time complexity:



b) Binary Search Algorithm:



step 1+ start
step 2+ create an array arr[100] and variables
size and key
step 3+ enter size of the array
step 4+ enter array elements & print
step 5+ sort the array using Bubble sort &
print sorted array.
step 6+ enter the element to be searched &
store in key.
step 7+ set low=0
Right = size-1
And calculate $mid = (low + high) / 2$
step 8+ compare key with arr[mid]
→ if $key == arr[mid]$ → element found
→ if $key < arr[mid]$ → set $high = mid - 1$
→ if $key > arr[mid]$ → set $low = mid + 1$
→ Repeat until $low \leq high$
step 9+ if $low > high$, print element not found

Program:

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n, target;
```

```
    cout << "Enter number of elements: ";
    cin >> n;
```

Name: D.himabindu
Enrollment number :
92400118934



```
int arr[100];

cout << "Enter elements in sorted order: ";
for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

cout << "Enter element to search: ";
cin >> target;

int low = 0;
int high = n - 1;
bool found = false;

while (low <= high) {
    int mid = low + (high - low) / 2;

    if (arr[mid] == target) {
        cout << "Element found at index " << mid << endl;
        found = true;
        break;
    }
    else if (arr[mid] < target) {
        low = mid + 1;
    }
    else {
        high = mid - 1;
    }
}

if (!found) {
    cout << "Element not found" << endl;
}

return 0;
}
```



Output:

```
Enter number of elements: 6
Enter elements in sorted order: 5
1
3
6
8
9
Enter element to search: 3
Element found at index 2
```

```
=== Code Execution Successful ===
```

Time complexity:



```
int low, high, mid; — 1
low = 0; — 1
high = size - 1; — 1
while (low <= high) — logn + 1
{
    mid = (low + high) / 2; — logn
    if (key == arr[mid]) — logn
    {
        cout << "element found"; — 1
        return; — 1
    }
    else if (key < arr[mid]) — logn
    {
        high = mid - 1; — logn
    }
    else
    {
        low = mid + 1 — logn
    }
}
```

$$\Rightarrow 1 + 1 + 1 + \log n + 1 + \log n + \log n + 1 + 1 + \log n + \log n$$

$$= \cancel{8} \log n + \cancel{8}$$

$$= \log n$$

$$= O(\log n)$$

Best case

$$O(1)$$

Average case

$$O(\log n)$$

Worst case

$$O(\log n)$$

Result:

As per the experiment to Implementation and Time analysis of linear and binary search algorithm.

Output was verified successfully according to the given experiment.

Name:D.himabindu
Enrollment number :
92400118934